

AI Capstone: Project 1 Report

110550170 戚維凌

March 2024

1. Introduction

The recent Meichu-Competition has just concluded, and while many people were enjoying the exciting volleyball matches, they often got confused by the gestures made by the referees on the court. These gestures indicate various aspects of the game, including which team scores, who serves, net-touching or ball out of bounds, and even special volleyball rules such as rotation errors. Watching the Meichu-Competition reminded us of this common confusion. Therefore, I am inspired to decide to seize this opportunity to create a dataset on this issue and collaborate with my classmate whose studentID is 110550034. Our goal is that training models to recognize referee gestures in real-time, helping people understand what is happening on the court instantly.

2. Dataset Description

In the beginning, we decided to perform image recognition. However, considering that using only images as one data would increase the risk of misclassification due to the similarity in many gestures' starting, middle, or ending movements, to achieve video classification, we choose to use short video clips and split them into multiple consecutive frames as a single data point, which will provide more valuable reference.

We selected six kinds of common gestures as the labels for this experiment, including double hit, holding, out of bounds, rotational fault, net-touching and touch out of bounds. Half of this dataset was extracted from online tutorial videos of referee gestures, while the other half videos was captured by us together.

In total, there are 229 short videos, with approximately 38 videos per gesture. Moreover, each video is divided into 30 (160x90) color frames, resulting in a total of 6,870 frames ([Appendix-A](#)).

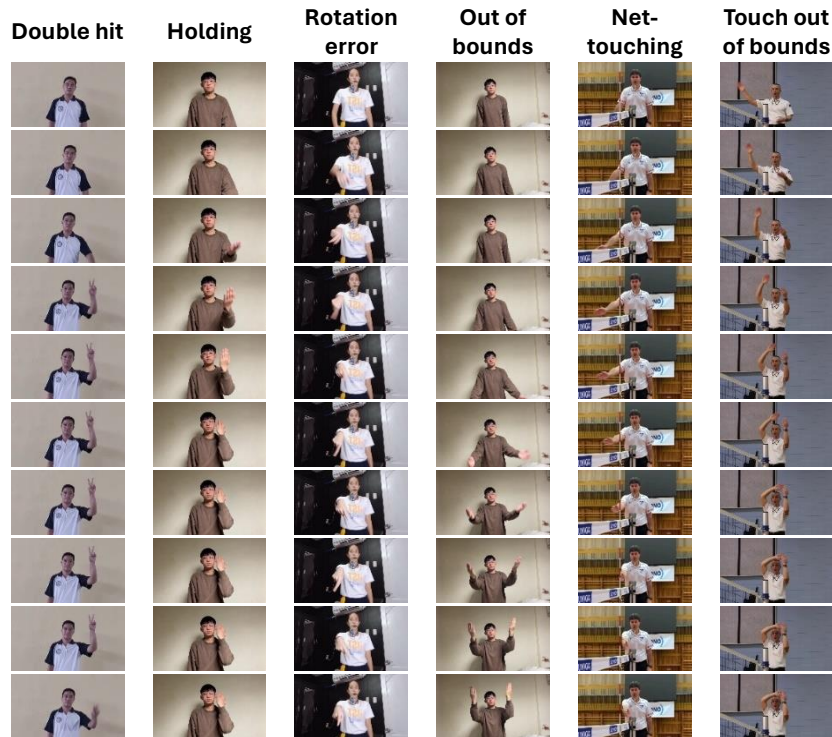


Figure 1: Ten consecutive frames of one data from six gestures

3. Method Description

For supervised learning, I use SVM classifier from scikit-learn and CNN built with TensorFlow framework.

3.1 SVM Classifier

Support Vector Machine (SVM) is a powerful supervised machine learning algorithm commonly used for classification and regression tasks. It works by finding the optimal hyperplane that separates different classes of data points in a high-dimensional space. As a result of its well performance on high-dimensional data classification and the fact that it doesn't require large amounts of data, I consider that it's suitable to be used as the training model for this project. Therefore, I utilize the pre-built SVM model from scikit-learn python library.

3.1.1 Data Preprocessing

Since our dataset consists of frames sliced from videos, and SVM requires two-dimensional input format, directly flattening and concatenating each frame into a one-dimensional array would result in greatly large data and too much noise and irrelevant features being considered by the model. Therefore, I utilized functions from OpenCV to extract optical flow as the feature extraction step before training the model ([Appendix-B](#)).

Optical flow refers to the pattern of apparent motion of objects, caused by the relative motion between the camera and the object. It represents the displacement of image features between consecutive frames in a sequence of video frames. In other words, optical flow describes how pixels move from one frame to another, providing valuable information about the motion and dynamics of objects in a scene. Simply, it means the information about the object motion. Why I choose to extract optical flow as a feature extraction method is because optical flow focuses only on motion, without considering colors or other factors. This advantage helps the model reduce unnecessary information more effectively from our dataset ([Appendix-C](#)).

3.1.2 Experiment and Discussion

I use linearSVM as the training model. Initially, I divided each video data point from the original dataset into 10 frames and 30 frames for accuracy comparison. Before experiment, I believed that dividing them into 10 frames is good enough to extract the features from gesture motions. Just as I thought, after observing the accuracy and confusion matrix, I found that the differences were not significant indeed. With cross validation, the average accuracy of 10 frames is 50.7% and 30 frames is 48.9%. Both are relatively low is due to the amount of original dataset is insufficiently and not comprehensive.

Divided into 10 frames

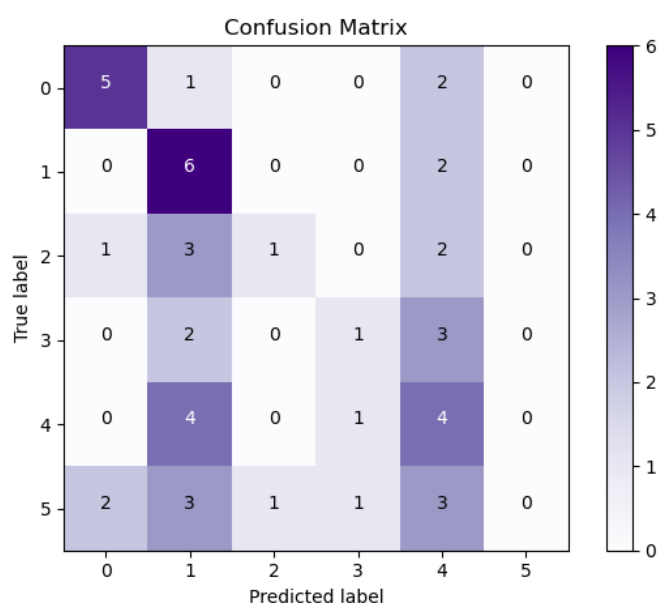


Figure 2: Average confusion matrix of each data point divided into 10 frames

Divided into 30 frames

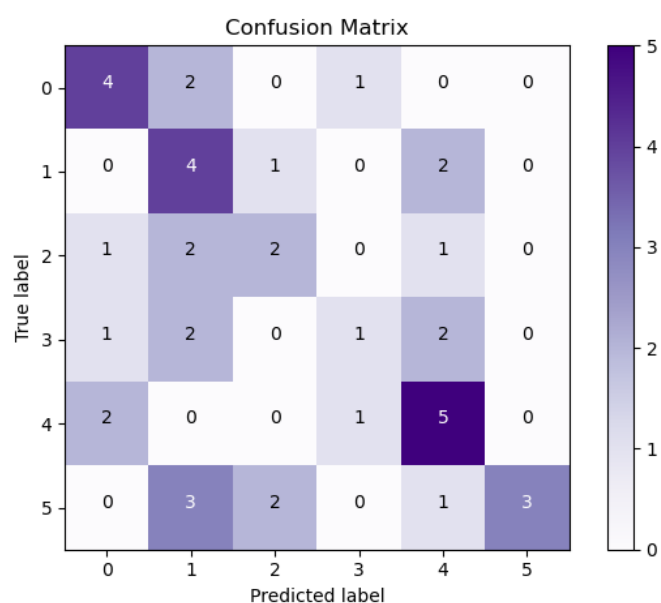


Figure 3: Average confusion matrix of each data point divided into 30 frames

However, when I used Video Data Augmentation (VIDAUG) python library to do augment dataset to 200 data points per category, including random horizontal flip, random shear and random translate ([Appendix-D](#)). Just as before, I observed by splitting the data into 10 frames and 30 frames. I also performed cross-validation as before, but this time there were slight differences in accuracy, the accuracy of 10 frames is 75.4% and 30 frames is 91.6%, being able to tell from the confusion matrix as well. The amount of training data has quite an impact on SVM classifier.

Divided into 10 frames
after augmented.

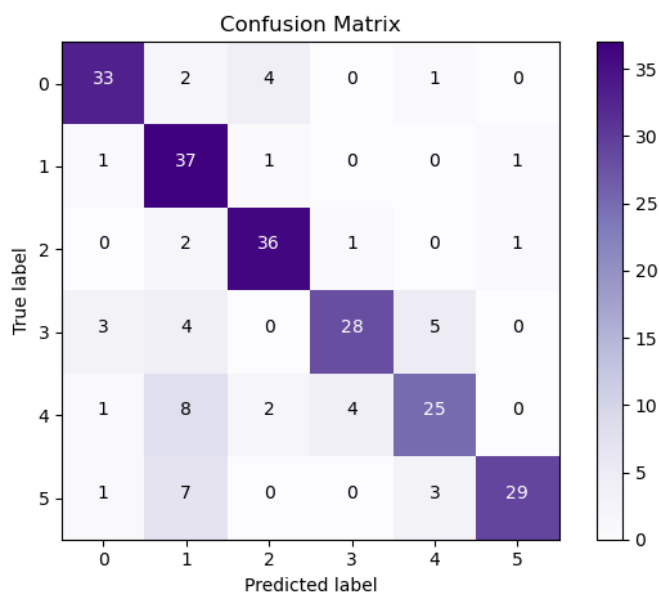


Figure 4: Average confusion matrix of each data point divided into 10 frames after augmented

Divided into 30 frames
after augmented.

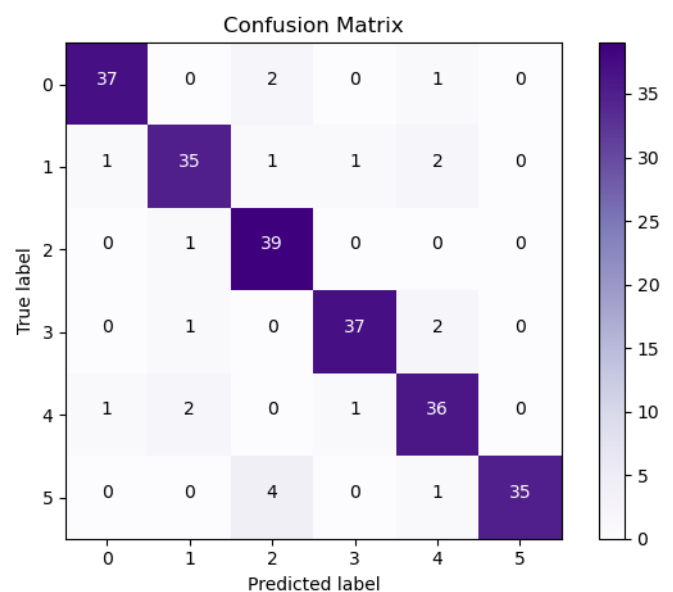


Figure 5: Average confusion matrix of each data point divided into 30 frames after augmented

Before above experiment, I had ever revised the dataset since I observed something that does not make sense. When testing the dataset that was augmented and divided into 30 frames, I noticed that one kind of gesture category was inaccurate compared to others. I found that there were some data from the rotation error gesture too confusing to be recognize. To be precise, the rotation error gesture should be done with only the wrist rotating not the whole arm rotating, which may cause model to misclassify the one-arm-movement gesture such as double hit, holding and net-touching.

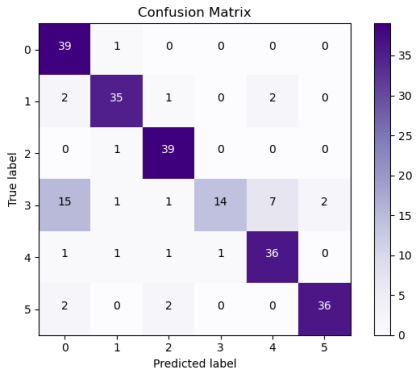


Figure 6: Misclassified to double hit

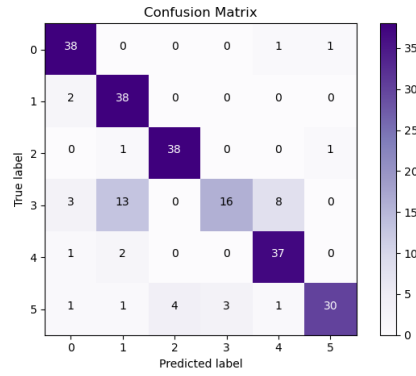


Figure 7: Misclassified to holding

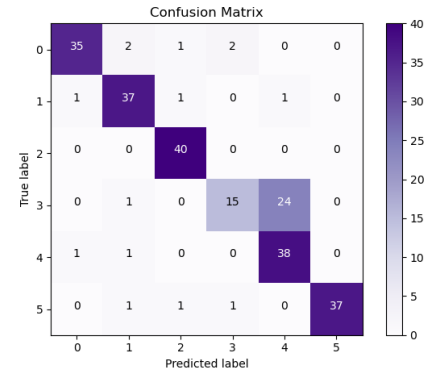


Figure 8: Misclassified to net-touching

3.2 Conv3D NN

Convolutional 3D Neural Network (Conv3D NN) is a type of three-dimensional convolutional neural network (CNN) layer designed for processing three-dimensional data, particularly video data. Conv3D applies a series of filters across the three dimensions of the input data, width, height, and time to extract features with the filters sliding along all three dimensions of the data to capture consecutive features such as motion, which is actually well-suited for my dataset. Therefore, I referred to the model built with TensorFlow's Keras by the author as the neural network architecture for training.

3.2.1 Data Preprocessing

This Conv3D NN starts with a 3D convolutional layer with 16 filters and a 5x5x5 filter size. Second, a 3D max pooling layer with a pool size of 2x2x2 is applied before batch normalization is performed to normalize the activations. The next two 3D convolutional layers are similar to the first layer. The difference lies in the second layer, which now has 32 filters and a 3x3x3 filter size. The pool size is changed to 1x2x2. The third layer continues from the second layer and is modified to have 64 filters. The final fully connected layer uses a SoftMax activation function to output probabilities for each gesture class.

I created a generator for data preprocessing and batching. Data preprocessing includes resizing the original data and normalizing each pixel additionally for complicated architecture of Conv3D ([Appendix-E](#)).

3.2.2 Experiment and Discussion

In the experiments this part, I will use the configuration that

- Epochs: 30
- Loss Function: SoftMax
- Optimizer: Adam
- Batch Size: 16

Since the issues on the dataset were found and corrected during the SVM experiments, the following experiment will focus on the scenario with the corrected dataset. Similarly, the accuracy differences between the number of frames from each data point will be observed in the beginning. This time I divided the data into 15 frames and 30 frames with original dataset not augmented. The cross validation average accuracy of splitting each data into 15 frames is 30.5% and 30 frames is 34.9% though both their loss did not converge. The amount of data is not enough for neural network to extract significantly different features or overfit.

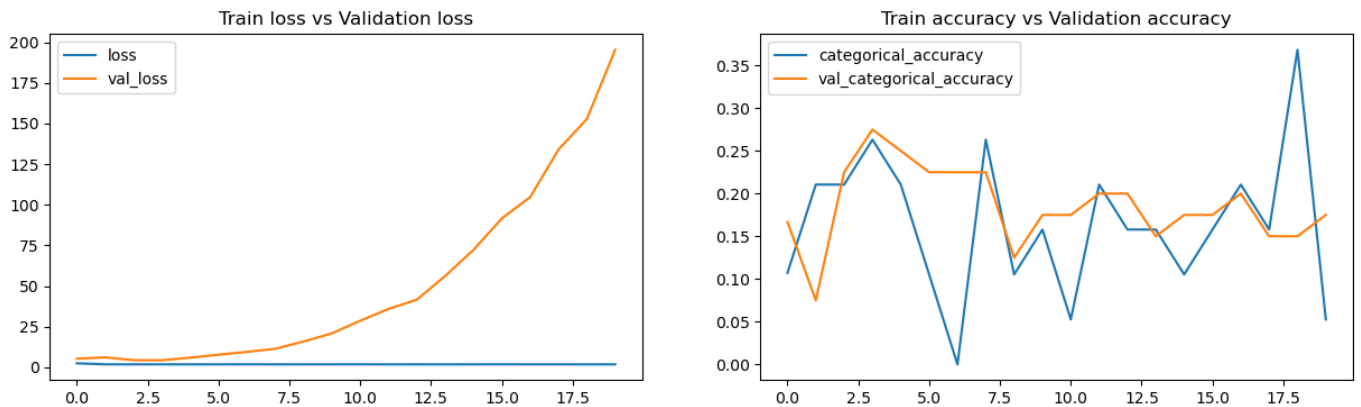


Figure 9: Model history curve with each data point divided into 15 frames

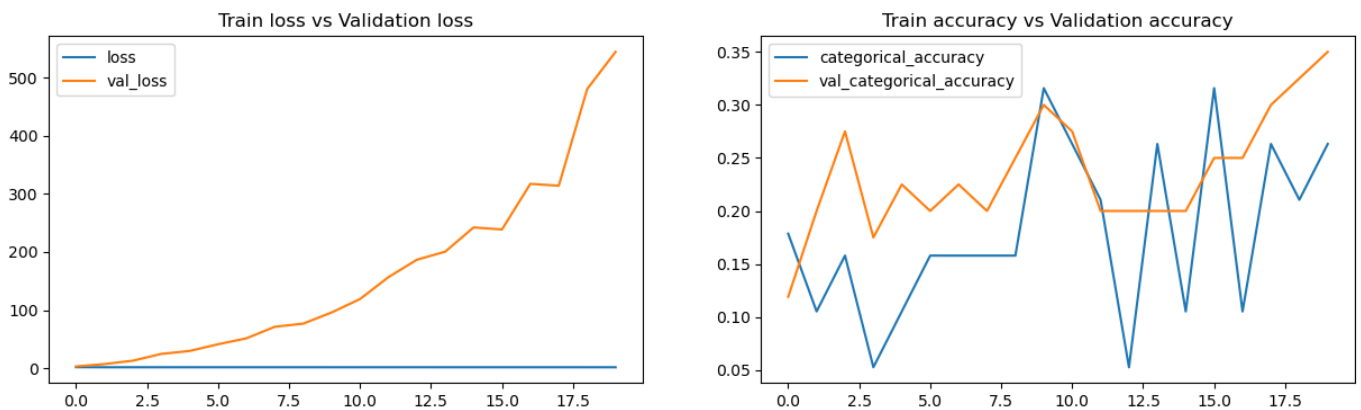


Figure 10: Model history curve with each data point divided into 30 frames

Next, I attempted to use the augmented dataset with cross-validation and splitting it into 15 frames and 30 frames again. The cross validation average accuracy about splitting each data into 15 frames is 85.6% and 30 frames is 83.7%.

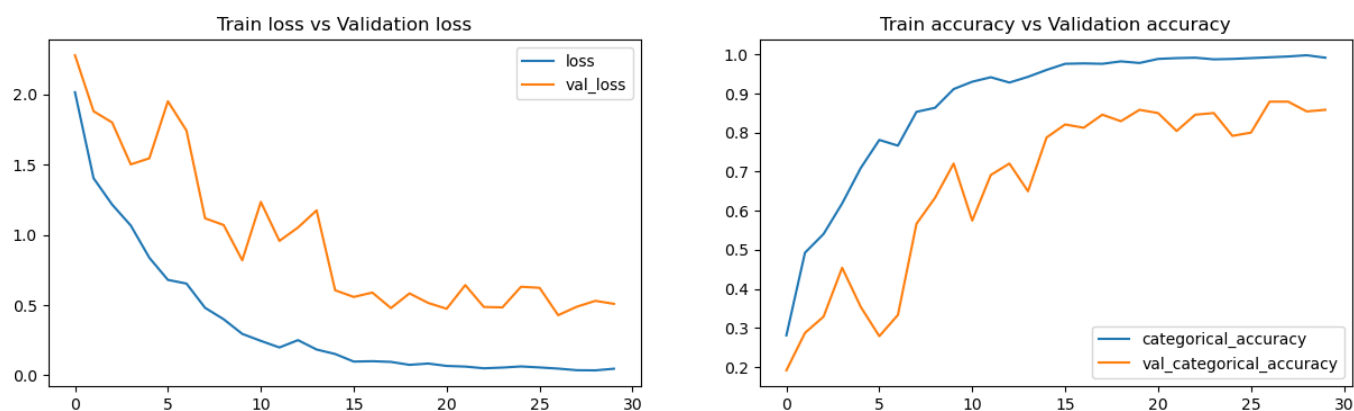


Figure 11: Model history curve with each data point divided into 15 frames

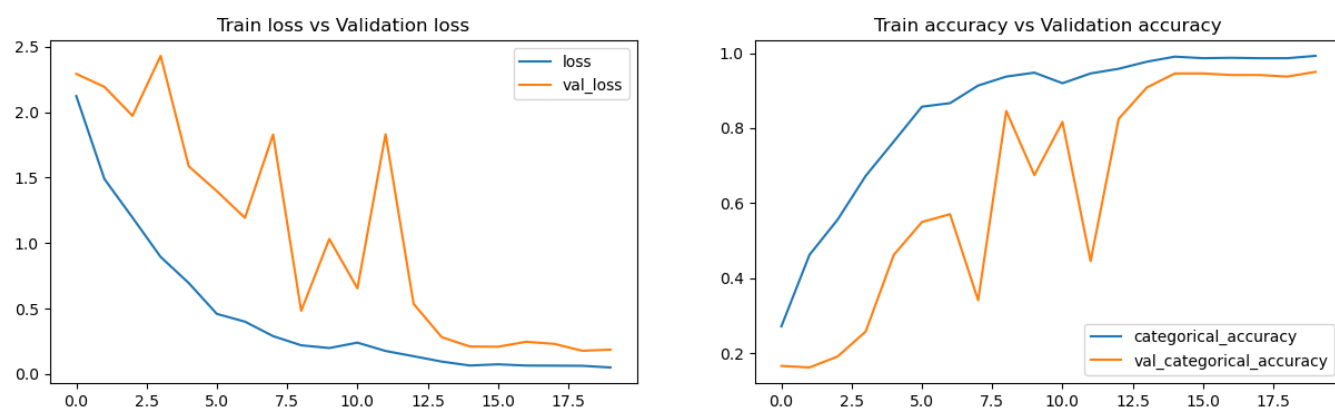


Figure 12: Model history curve with each data point divided into 30 frames

Surprisingly, the accuracy did not differ significantly between the two. I expected that the results should be similar to SVM. However, it becomes apparent that SVM relies directly on the information represented by the variations within consecutive frames for video classification. It requires accurate and sufficient consecutive data, especially when using linear SVM. The hyperplane may not ignore important feature between the frames. On the other hand, Conv3D can extract time and spatial features from video sequences, which are typically related to the global structure of the video rather than directly dependent on the number of frames. For the reason that the difference between the two on SVM is more obviously than it on Conv3D NN.

For unsupervised learning, I use K-means classifier from scikit-learn python library.

3.3 K-means Classifier

K-means clustering is a popular unsupervised learning algorithm used for partitioning a given dataset into K distinct, non-overlapping clusters by minimizing the within-cluster variance, or the sum of squared distances between data points and their corresponding cluster centroids.

3.3.1 Data Preprocessing

If the data's information is directly combined into a one-dimensional array as the model input, it will become excessively large, making the model training slow. To better adapt my dataset to the input requirements of K-means, I utilize the pre-trained model VGG16 from TensorFlow Keras to extract important features and reduce the input dimensionality for the k-means model. VGG16 is an improved CNN, and I only use its top layers as a feature extraction tool. I further perform the PCA method from scikit-learn python library to retain only the most significant parts of the data before putting it into the model ([Appendix-F](#)).

3.3.2 Experiment and Discussion

In this section, to determine the performance from experiment, first, I set the number of clusters found from k-means model to 36. Next, I compare the cluster numbers generated by k-means with the true labels. For each different cluster number, I calculate which gesture is most frequently classified into that cluster. Then, I consider this cluster number as the cluster for that gesture. After pairing all clusters and gestures, we can know the accuracy and confusion matrix ([Appendix-G](#)).

I still observe the results using different numbers of frames with the original dataset initially. The accuracy of 15 frames is 47.4% and 30 frames is 43.5%. The reason why they do not have large differences is same as section 3.2.2. Performing VGG16 to extract the features just like using CNN to do that. They both can extract true features from video sequences, which are typically related to the global structure of the video rather than directly dependent on the number of frames.

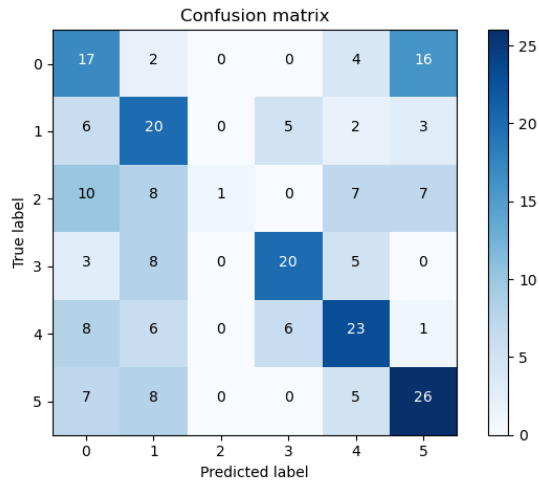


Figure 13: Average confusion matrix of each data point divided into 10 frames

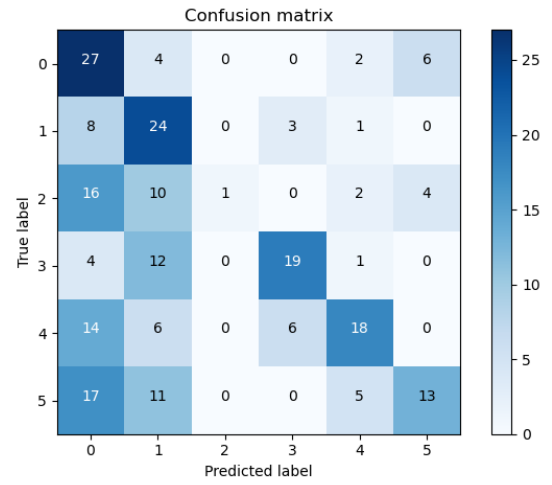


Figure 14: Average confusion matrix of each data point divided into 30 frames

When I use augmented dataset, the accuracy of 15 frames is 46.6% and 30 frames is 47.0%. From this, it can be inferred that simply relying on data augmentation of the original dataset to increase the amount of data introduce too much noise or insufficient feature coverage. Unfortunately, k-means model is sensitive to abnormal data points and noises. As a result, the accuracy does not raise.

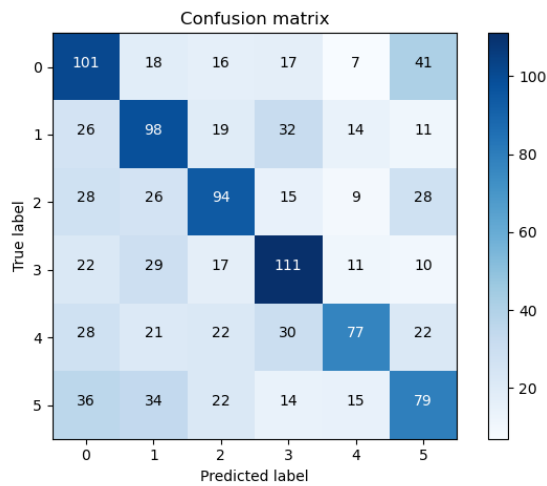


Figure 15: Average confusion matrix of each data point divided into 10 frames

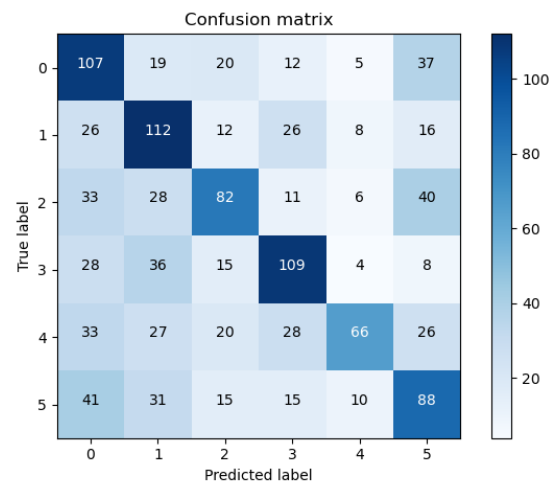


Figure 16: Average confusion matrix of each data point divided into 30 frames

4. Whole Discussion

Overall, the results of training and experiment with the two supervised learning models met my expectations. SVM places a strong emphasis on data integrity and the inclusiveness of features, whereas CNN is less affected by these factors. Moreover, both models are also sensitive to the amount of data available; increasing the size and breadth of the dataset can improve their accuracy. The quality of the dataset is also crucial, as incomplete or inaccurate data can have an impact. For example, during the SVM experiment, I found that several data points with rotation errors had inaccurate movements. After correction, the accuracy performs normally.

Before the experiment, I believed that unsupervised learning would not be suitable for our topic, and the experimental results also confirmed this. The first reason I think is that the training data has too much noise and is incomplete in collection. The second reason is that from the perspective of videos, features are distributed across three dimensions. Preprocessing such as PCA is needed, but the effect after PCA is not much better than expected. Moreover, the time spent is much higher than supervised learning, which prevents me from conducting more experiments to identify the issues. About future experiments, I will perform more experiments on unsupervised learning model, including more preprocessing and more powerful model to come up with the main problem.

5. References

- 5.1 [\[機器學習首部曲\] 支援向量機 SVM – PyInvest \(pyecontech.com\)](#)
- 5.2 [\[機器學習 ML NOTE\]Convolution Neural Network 卷積神經網路|Medium](#)
- 5.3 [Day19-Scikit-learn 介紹\(11\) K-Means](#)
- 5.4 [Gesture Recognition - Conv3D & Conv2D+RNN \(kaggle.com\)](#)
- 5.5 [Dense optical flow with Python using OpenCV. | Medium](#)
- 5.6 [K-Means Clustering for Image Classification |Medium](#)
- 5.7 [Day 09 : CNN 經典模型應用](#)
- 5.8 [How to do video data augmentation using the Python VIDAUG library | Medium](#)
- 5.9 [vidaug: Effective Video Augmentation Techniques for Training Convolutional Neural Networks](#)

6. Appendix

Appendix-A: Turn Video to Frames Code

```
1. import numpy as np
2. import cv2
3. import os
4.
5. def extract_frames(video_path, output_dir):
6.
7.     video = cv2.VideoCapture(video_path)
8.     os.makedirs(output_dir, exist_ok=True)
9.
10.    cnt = 1
11.    # get the number of frames in the video
12.    frame_count = int(video.get(cv2.CAP_PROP_FRAME_COUNT) / 30)
13.    if frame_count == 0:
14.        frame_count = 1
15.
16.    success, frame = video.read()
17.    if success: frame = cv2.resize(frame, (160, 90))
18.
19.    last_frame = frame
20.    # loop through the video and save the frames
21.    while success:
22.        # limit the number of frames to 30
23.        if cnt > 30:
24.            break
25.        output_path = os.path.join(output_dir, f"frame_{str(cnt).zfill(3)}.jpg")
26.        cv2.imwrite(output_path, frame)
27.        cnt += 1
28.
29.        for _ in range(frame_count):
30.            success, frame = video.read()
31.            if not success:
32.                break
33.            frame = cv2.resize(frame, (160, 90))
34.            last_frame = frame
35.
36.    # if the video is less than 30 frames, duplicate the last frame
37.    if cnt <= 30:
38.        for i in range(cnt, 31):
39.            output_path = os.path.join(output_dir, f"frame_{str(i).zfill(3)}.jpg")
40.            cv2.imwrite(output_path, last_frame)
41.
42.
43. if __name__ == "__main__":
44.     dataset_dir = "./dataset"
45.     # loop through the dataset and split the video to frames
46.     for catagory in os.listdir(dataset_dir):
47.         cnt = 1
48.         catagory_path = os.path.join(dataset_dir, catagory)
49.         for video in os.listdir(catagory_path):
50.             video_path = os.path.join(catagory_path, video)
51.             if os.path.isdir(video_path):
52.                 continue
53.             extract_frames(video_path, os.path.join(catagory_path,
54. f"{catagory}_{str(cnt).zfill(4)}"))
55.             print("done")
56.             cnt += 1
```

Appendix-B: Optical Flow Extracting Code

```
1. def extract_optical_flow(frames):
2.     features = []
3.     # initial the previous frame
4.     prev_frame = cv2.cvtColor(frames[0], cv2.COLOR_BGR2GRAY)
5.
6.     for frame in frames[1:]:
7.         # turn the frame into gray scale
8.         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
9.
10.        # calculate the optical flow
11.        flow = cv2.calcOpticalFlowFarneback(prev_frame, gray, None, 0.5, 3, 15, 3, 5,
12.        1.2, 0)
13.
14.        # combine the optical flow in x and y direction
15.        flow_feature = flow.reshape(-1)
16.        features.append(flow_feature)
17.
18.        # update the previous frame
19.        prev_frame = gray
20.
21.    return np.array(features)
```

Appendix-C: SVM Classifier Code

```
1. import cv2
2. import os
3. import numpy as np
4. from sklearn.svm import SVC
5. from sklearn.metrics import accuracy_score, confusion_matrix
6. import csv
7. import matplotlib.pyplot as plt
8. import itertools
9.
10. num_labels = 6
11.
12. def load_frames(vid_path):
13.     frames = []
14.
15.     imgs = os.listdir(vid_path)
16.     for i in range(len(imgs)):
17.         img_path = os.path.join(vid_path, imgs[i])
18.         frame = cv2.imread(img_path)
19.
20.         frames.append(frame)
21.
22.     return frames
23.
24.
25. def extract_optical_flow(frames):
26.     features = []
27.     # initial the previous frame
28.     prev_frame = cv2.cvtColor(frames[0], cv2.COLOR_BGR2GRAY)
29.
30.     for frame in frames[1:]:
```

```

31.         # turn the frame into gray scale
32.         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
33.
34.         # calculate the optical flow
35.         flow = cv2.calcOpticalFlowFarneback(prev_frame, gray, None, 0.5, 3, 15, 3, 5,
1.2, 0)
36.
37.         # combine the optical flow in x and y direction
38.         flow_feature = flow.reshape(-1)
39.         features.append(flow_feature)
40.
41.         # update the previous frame
42.         prev_frame = gray
43.
44.     return np.array(features)
45.
46.
47. def preprocess(path, csv_path):
48.     vids = [[] for _ in range(num_labels)]
49.     with open(csv_path, 'r') as file:
50.         rows = csv.reader(file)
51.         for row in rows:
52.             vids[int(row[2])].append(os.path.join(row[1], row[0]))
53.
54.     features = []
55.     labels = []
56.
57.     for l in range(num_labels):
58.         for i in range(len(vids[l])):
59.             vid_path = os.path.join(path, vids[l][i])
60.             frames = load_frames(vid_path)
61.             optical_flow_features = extract_optical_flow(frames)
62.             features.append(optical_flow_features)
63.             labels.append(l)
64.
65.     return np.array(features), np.array(labels)
66.
67. # load data
68. X_train, y_train = preprocess('./dataset_aug_ori', './train_ori_4.csv')
69. print(X_train.shape, y_train.shape)
70. X_test, y_test = preprocess('./dataset_aug_ori', './val_ori_4.csv')
71. print(X_test.shape, y_test.shape)
72.
73. # turn the three-dimensional feature array into a two-dimensional one
74. X_train_flat = X_train.reshape((X_train.shape[0], -1))
75. X_test_flat = X_test.reshape((X_test.shape[0], -1))
76.
77. svm_model = SVC(kernel='linear', C=0.1)
78.
79. # train the model
80. svm_model.fit(X_train_flat, y_train)
81.
82. # predict the test data
83. y_pred = svm_model.predict(X_test_flat)
84.
85. # calculate the accuracy
86. accuracy = accuracy_score(y_test, y_pred)
87. print(f'Accuracy: {accuracy}')
88.
89. # confuson matrix
90. cnf_matrix_sklearn = confusion_matrix(y_test, y_pred)
91. print(cnf_matrix_sklearn)

```

Appendix-D: Video Augmentation Code

```
1. import os
2. import cv2
3. import numpy as np
4. import vidaug.augmentors as va
5. import shutil
6.
7. dataset_dir = './dataset'
8. output_dir = './dataset_aug_new'
9. shutil.copytree(dataset_dir, output_dir) # copy the dataset to the output directory
10. sometimes = lambda aug: va.Sometimes(0.5, aug) # Used to apply augmentor with 50%
    probability
11.
12. # Loop through the dataset and augment the videos
13. for category in os.listdir(output_dir):
14.     category_path = os.path.join(output_dir, category)
15.     done = False
16.     while not done:
17.         for video in os.listdir(category_path):
18.             video_path = os.path.join(category_path, video)
19.             if not os.path.isdir(video_path):
20.                 continue
21.
22.             folder_i = len([file for file in os.listdir(category_path) if
os.path.isdir(os.path.join(category_path, file))]) + 1
23.             if folder_i > 200: # limit the number of videos to 200
24.                 done = True
25.                 print(category)
26.                 break
27.
28.             folder_name = f"{category}_{str(folder_i).zfill(4)}"
29.             os.makedirs(os.path.join(category_path, folder_name), exist_ok=True)
30.             frames = []
31.             for frame in os.listdir(video_path):
32.                 frame_path = os.path.join(video_path, frame)
33.                 img = cv2.imread(frame_path, cv2.IMREAD_COLOR)
34.                 frames.append(img)
35.
36.             # Define the augmentation sequence
37.             seq = va.Sequential([sometimes(va.HorizontalFlip()),
38.                                 sometimes(va.RandomShear(-0.1, 0.1)),
39.                                 sometimes(va.RandomTranslate(5, 5))])
40.
41.             #augment the frames
42.             img_aug = seq(frames)
43.             for frame in img_aug:
44.                 # output the video
45.                 frame_i = len([file for file in
os.listdir(os.path.join(category_path, folder_name))])
46.                 cv2.imwrite(os.path.join(os.path.join(category_path, folder_name),
f"frame_{str(frame_i).zfill(3)}.jpg"), frame)
```

Appendix-E: Conv3D Neural Network Code

```
1. import numpy as np
2. import os
3. from skimage.transform import resize
4. from imageio import imread
5. import datetime
6. import tensorflow as tf
7. from keras.models import Sequential
8. from keras.layers import Dense, Flatten, Flatten, BatchNormalization, Dropout
9. from keras.layers.convolutional import Conv3D, MaxPooling3D
10.
11. # turn image to tensor with n_frames
12. def getImgTensor(n_frames):
13.     img_idx = np.round(np.linspace(0, 29, n_frames)).astype(int)
14.     return [img_idx, 90, 160, 3]
15.
16. # normalize image
17. def normalization(image):
18.     return image/255.0
19.
20. # get batch data
21. def getBatchData(source_path, t, batch, batch_size, img_tensor):
22.     [x,y,z] = [len(img_tensor[0]),img_tensor[1], img_tensor[2]]
23.     img_idx = img_tensor[0]
24.     batch_data = np.zeros((batch_size,x,y,z,3)) # x is the number of images you use
        for each video, (y,z) is the final size of the input images and 3 is the number of
        channels RGB
25.     batch_labels = np.zeros((batch_size, gesture_num)) # batch_labels is the one hot
        representation of the output
26.     for folder in range(batch_size): # iterate over the batch_size
27.         imgs = os.listdir(source_path+'/'+ t[folder +
        (batch*batch_size)].split(',')[0]) # read all the images in the folder
28.         for idx,item in enumerate(img_idx): # Iterate iver the frames/images of a
        folder to read them in
29.
30.             image = imread(source_path+'/'+ t[folder +
        (batch*batch_size)].strip().split(',')[0]+'/' +imgs[item]).astype(np.float32)
31.
32.             # normalizing images for 3 channels
33.             batch_data[folder,idx,:,:,0] = normalization(image[:, :, 0])
34.             batch_data[folder,idx,:,:,1] = normalization(image[:, :, 1])
35.             batch_data[folder,idx,:,:,2] = normalization(image[:, :, 2])
36.
37.             batch_labels[folder, int(t[folder +
        (batch*batch_size)].strip().split(',')[2]]) = 1
38.     return batch_data, batch_labels
39.
40. # generator each batch data
41. def generator(source_path, folder_list, batch_size, img_tensor):
42.     while True:
43.         # t is random list of folder_list
44.         t = np.random.permutation(folder_list)
45.         num_batches = int(len(folder_list)/batch_size) # calculate the number of
        batches
46.         for batch in range(num_batches): # we iterate over the number of batches
47.             yield getBatchData(source_path, t, batch, batch_size, img_tensor)
48.             # checking if any remaining batches are there or not
49.             if len(folder_list)%batch_size != 0:
50.                 # updated the batch size and yield
51.                 batch_size = len(folder_list)%batch_size
```



```

52.         yield getBatchData(source_path, t, batch, batch_size, img_tensor)
53.
54. # Set the configuration and source path
55. train_doc = np.random.permutation(open('./train.csv').readlines())
56. val_doc = np.random.permutation(open('./val.csv').readlines())
57. train_path = './train'
58. val_path = './val'
59. gesture_num = 6
60. curr_dt_time = datetime.datetime.now()
61. num_train_sequences = len(train_doc)
62. num_val_sequences = len(val_doc)
63. n_frames = 15
64. num_epochs = 30
65. batch_size = 8
66.
67. # define image tensor size and generator
68. img_tensor = getImgTensor(n_frames)
69. train_generator = generator(train_path, train_doc, batch_size, img_tensor)
70. val_generator = generator(val_path, val_doc, batch_size, img_tensor)
71.
72. # calculate steps per epoch and validation steps
73. if (num_train_sequences%batch_size) == 0:
74.     steps_per_epoch = int(num_train_sequences/batch_size)
75. else:
76.     steps_per_epoch = (num_train_sequences//batch_size) + 1
77.
78. if (num_val_sequences%batch_size) == 0:
79.     validation_steps = int(num_val_sequences/batch_size)
80. else:
81.     validation_steps = (num_val_sequences//batch_size) + 1
82.
83. inputShape = (len(img_tensor[0]), img_tensor[1], img_tensor[2], img_tensor[3])
84. # bulid model
85. model1 = Sequential([
86.     # 1st Conv3D layer group
87.     Conv3D(16, (5, 5, 5), activation='relu', input_shape=inputShape),
88.     MaxPooling3D((2, 2, 2), padding='same'),
89.     BatchNormalization(),
90.
91.     # 2nd Conv3D layer group
92.     Conv3D(32, (3, 3, 3), activation='relu'),
93.     MaxPooling3D(pool_size=(1,2,2), padding='same'),
94.     BatchNormalization(),
95.
96.     # 3rd Conv3D layer group
97.     Conv3D(64, (3, 3, 3), activation='relu'),
98.     MaxPooling3D(pool_size=(1,2,2), padding='same'),
99.     BatchNormalization(),
100.
101.     Flatten(),
102.     Dense(128, activation='relu'),
103.     BatchNormalization(),
104.     Dropout(0.25),
105.
106.     Dense(64, activation='relu'),
107.     BatchNormalization(),
108.     Dropout(0.25),
109.
110.     # output layer with softmax activation
111.     Dense(gesture_num, activation='softmax')
112. ], name="conv_3d1")
113. # compile model
114. model1.compile(optimizer=tf.keras.optimizers.Adam(),
    loss='categorical_crossentropy', metrics=['categorical_accuracy'])

```

```

115. print(model1.summary())
116. # train model
117. model1_history = model1.fit_generator(train_generator,
    steps_per_epoch=steps_per_epoch, epochs=num_epochs, verbose=1,
118.     validation_data=val_generator,
119.     validation_steps=validation_steps, class_weight=None, workers=1,
    initial_epoch=0)

```

Appendix-F: K-means and VGG16 Feature Extraction Code

```

1. import cv2
2. import os
3. import numpy as np
4. from keras.applications.vgg16 import VGG16, preprocess_input
5. from keras.models import Model
6. from sklearn.decomposition import PCA
7. from sklearn.cluster import KMeans
8.
9. # VGG16 model, with weights pre-trained on ImageNet. Only include the convolutional
    part.
10. base_model = VGG16(weights='imagenet', include_top=False)
11. model = Model(inputs=base_model.input,
    outputs=base_model.get_layer('block4_pool').output)
12.
13. # Load dataset
14. dataset_dir = './dataset_aug_new'
15.
16. dataset = []
17. for category in os.listdir(dataset_dir):
18.     shape = (1, len(os.listdir(os.path.join(dataset_dir, category))))
19.
20.     category_path = os.path.join(dataset_dir, category)
21.     for index, video in enumerate(os.listdir(category_path)):
22.         video_path = os.path.join(category_path, video)
23.         if not os.path.isdir(video_path):
24.             continue
25.
26.         frames = []
27.         cnt = -1
28.         for frame in os.listdir(video_path):
29.             cnt += 1
30.             frame_path = os.path.join(video_path, frame)
31.             img = cv2.imread(frame_path)
32.             img = img / 255.0
33.             img = np.expand_dims(img, axis=0) # boarden the dimension
34.             img = preprocess_input(img) # preprocess the image
35.             frames.append(img)
36.
37.         dataset.append(frames)
38.
39. dataset = np.array(dataset)
40.
41. # Extract features
42. videos_features = []
43. for data in dataset:
44.     features = []
45.     for frame in data:
46.         feature = model.predict(frame) # extract feature from VGG16

```

```

47.     features.append(feature)
48.     videos_features.append(np.concatenate(features, axis=0))
49.
50. data = np.array(videos_features)
51. # Reshape the data
52. X = data.reshape((data.shape[0], -1))
53.
54. # PCA
55. pca = PCA(n_components=1000)
56. X = pca.fit_transform(X)
57. print(X.shape)
58.
59. # train kmeans model
60. kmeans = KMeans(n_clusters=128)
61. kmeans.fit(X)
62. Y = kmeans.labels_

```

Appendix-G: Calculate Accuracy and Confusion Matirx Code

```

1. import itertools
2. import numpy as np
3. import matplotlib.pyplot as plt
4. from sklearn.metrics import confusion_matrix, accuracy_score
5.
6. # define plot function
7. def plot_confusion_matrix(cm, classes,
8.                           normalize=False,
9.                           title='Confusion matrix',
10.                          cmap=plt.cm.Blues):
11.     if normalize:
12.         cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
13.
14.     plt.imshow(cm, interpolation='nearest', cmap=cmap)
15.     plt.title(title)
16.     plt.colorbar()
17.     tick_marks = np.arange(len(classes))
18.     plt.xticks(tick_marks, classes)
19.     plt.yticks(tick_marks, classes)
20.
21.     fmt = '.2f' if normalize else 'd'
22.     thresh = cm.max() / 2.
23.     for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
24.         plt.text(j, i, format(cm[i, j], fmt),
25.                  horizontalalignment="center",
26.                  color="white" if cm[i, j] > thresh else "black")
27.
28.     plt.ylabel('True label')
29.     plt.xlabel('Predicted label')
30.     plt.tight_layout()
31.     plt.savefig('result/k-means/output_aug_30frames.png')
32.
33. def retrieve_info(cluster_labels, y_train):
34.     # Initializing
35.     reference_labels = {}
36.     # For loop to run through each label of cluster label
37.     for i in range(len(np.unique(Y))):
38.         index = np.where(cluster_labels == i)[0] # get the index of elements in the
            cluster

```

```
39.         if len(index) > 0:
40.             num = np.bincount(y_train[index]).argmax() # calculate the mode of the
class in the cluster
41.             reference_labels[i] = num
42.         else:
43.             reference_labels[i] = -1 # deal with the empty cluster
44.     return reference_labels
45.
46. # generate corresponding labels
47. y_test = [0]*200 + [1]*200 + [2]*200 + [3]*200 + [4]*200 + [5]*200
48. y_test = np.array(y_test)
49. reference_labels = retrieve_info(Y, y_test)
50. number_labels = np.random.rand(len(Y))
51. # replace the cluster labels with the original labels
52. for i in range(len(Y)):
53.     number_labels[i] = reference_labels[kmeans.labels_[i]]
54.
55. cnf_matrix_sklearn = confusion_matrix(y_test, number_labels)
56. acc = accuracy_score(y_test, number_labels)
57. print("accuracy score:", acc, '\n')
58. print(cnf_matrix_sklearn)
59. plot_confusion_matrix(cnf_matrix_sklearn, [0, 1, 2, 3, 4, 5], normalize=False,
title='Confusion matrix', cmap=plt.cm.Blues)
```